

BANK OF PUNJAB MANAGEMENT SYSTEM

1. Project Introduction

The Bank of Punjab (BOP) Management System is a desktop-based application developed using Java Swing and MySQL. The system is designed to streamline banking operations such as client registration, account management, and financial transactions (Deposits/Withdrawals). The application features a modern "Glassmorphism" UI with transparent inputs and a secure database backend.

2. System Architecture

The project follows a modular architecture to ensure scalability and maintainability:

- ✚ **Presentation Layer (GUI):** Built with Java Swing, featuring a consistent BaseFrame for a unified look across all screens.
- ✚ **Business Logic Layer:** Handles data validation and communication between the UI and the Database.
- ✚ **Data Layer (MySQL):** Stores all persistent data, including user credentials, client profiles, and transaction logs.

3. Features & Modules

3.1 User Authentication

A secure login system that authenticates users against the users table. The interface features a split-screen design with a professional banking image on one side and a dark-themed login form on the other.

3.2 Client & Account Management

- ✚ **Client Registration:** Allows bank staff to add or update client details (Name, Phone).
- ✚ **Account Opening:** Facilitates the creation of bank accounts linked to specific clients with initial balance settings.

3.3 Financial Transactions

The system supports automated transactions through:

- ✚ **Deposits:** Increases account balance and records a 'Deposit' entry.

- ✚ **Withdrawals:** Decreases account balance while ensuring the balance does not drop below zero via database constraints.

4. Database Design & Advanced Implementation

4.1 Tables and Relationships

The database consists of five core tables: users, client, account, and transactions. Relationships are maintained via Foreign Keys to ensure data integrity (e.g., deleting a client automatically handles their associated accounts).

4.2 Stored Procedures

To optimize performance, critical logic is handled within the database:

- ✚ **deposit_money(acc, amt):** Updates balance and logs the transaction.
- ✚ **withdraw_money(acc, amt):** Subtracts the amount and logs the transaction.

4.3 Triggers (Automation & Security)

- ✚ **check_balance:** A BEFORE UPDATE trigger that prevents any transaction that would result in a negative balance, raising an "Insufficient Balance" error.
- ✚ **after_transaction_insert:** Automatically adjusts the account balance whenever a new record is added to the transaction table.

4.4 Database Views

- ✚ **clientAccountsView:** A complex join that provides a comprehensive snapshot of clients and their current balances in a single virtual table.

5. Technical Requirements Fulfillment

Based on the project rubric, the following have been implemented:

- ✚ **GUI Components:** Use of text boxes, password fields, dropdowns, and custom panels.
- ✚ **Exception Handling:** Managed via Java try-catch blocks and SQL SIGNAL states.
- ✚ **Responsive UI:** Consistent layout across different forms using the BaseFrame template.

Following is sql queries.

```
CREATE DATABASE BankDB;
```

```
USE BankDB;
```

-- Tables

```
CREATE TABLE users (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    username VARCHAR(50) NOT NULL UNIQUE,
```

```
    password VARCHAR(50) NOT NULL
```

```
);
```

```
INSERT INTO users (username, password) VALUES
```

```
('admin', 'admin123'),('user1', 'pass1');
```

```
CREATE TABLE client (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
```

```
    phone VARCHAR(15)
```

```
);
```

```
INSERT INTO client (name, phone) VALUES
```

```
('Ali Khan', '03001234567'),('Sara Ahmed', '03007654321');
```

```
CREATE TABLE account (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    client_id INT NOT NULL,
```

```
    balance DECIMAL(15,2) DEFAULT 0,
```

```
    FOREIGN KEY (client_id) REFERENCES client(id) ON DELETE CASCADE
```

```
);
```

```
INSERT INTO account (client_id, balance) VALUES(1, 5000),(2, 10000);
```

```
DROP TABLE IF EXISTS transactions;
```

```
CREATE TABLE transactions (  
    tx_id INT AUTO_INCREMENT PRIMARY KEY,  
    acc_id INT,  
    amount DOUBLE,  
    type VARCHAR(20),  
    tx_date DATE  
);
```

-- STORED PROCEDURE

```
DELIMITER //  
  
CREATE PROCEDURE addTransaction(  
    IN p_account_id INT,  
    IN p_amount DECIMAL(15,2),  
    IN p_type VARCHAR(20)  
)  
  
BEGIN  
    INSERT INTO transaction(account_id, amount, type)  
VALUES (p_account_id, p_amount, p_type);  
  
END //  
  
DELIMITER ;
```

```
DELIMITER $$  
  
CREATE PROCEDURE deposit_money(  
    IN acc INT,  
    IN amt DOUBLE  
)  
  
BEGIN  
    UPDATE account
```

```
    SET balance = balance + amt

    WHERE id = acc;

    INSERT INTO transactions(acc_id,amount,type)

    VALUES(acc, amt, 'Deposit');

END$$

DELIMITER ;
```

```
DELIMITER $$

CREATE PROCEDURE withdraw_money(

    IN acc INT,

    IN amt DOUBLE

)

BEGIN

    UPDATE account

    SET balance = balance - amt

    WHERE id = acc;

    INSERT INTO transactions(acc_id,amount,type)

    VALUES(acc, amt, 'Withdraw');

END$$

DELIMITER ;
```

-- TRIGGER

```
DELIMITER //

CREATE TRIGGER after_transaction_insert

AFTER INSERT ON transaction

FOR EACH ROW

BEGIN

    IF NEW.type = 'Deposit' THEN
```

```

        UPDATE account SET balance = balance + NEW.amount WHERE id = NEW.account_id;

    ELSEIF NEW.type = 'Withdraw' THEN

        UPDATE account SET balance = balance - NEW.amount WHERE id = NEW.account_id;

    END IF;

END //

DELIMITER ;

DELIMITER $$

CREATE TRIGGER check_balance

BEFORE UPDATE ON account

FOR EACH ROW

BEGIN

    IF NEW.balance < 0 THEN

        SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Insufficient Balance';

    END IF;

END$$

DELIMITER ;

-- VIEW

CREATE VIEW clientAccountsView AS

SELECT c.id AS client_id, c.name, c.phone, a.id AS account_id,a.balance

FROM client c

JOIN account a ON c.id = a.client_id;

-- Check clients and accounts

SELECT * FROM clientAccountsView;

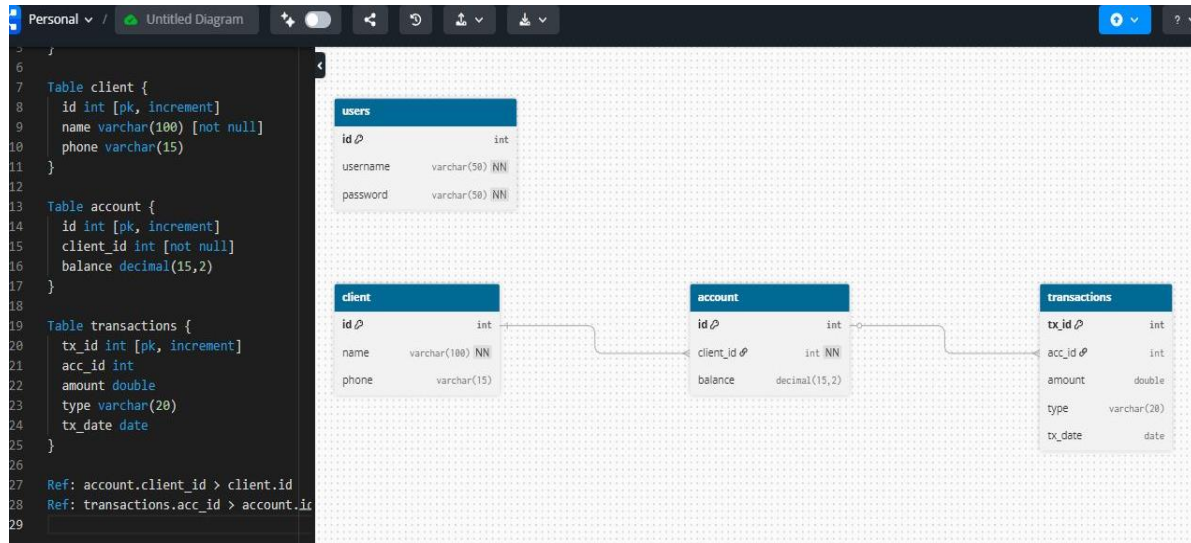
-- Check transactions

SELECT * FROM transaction;

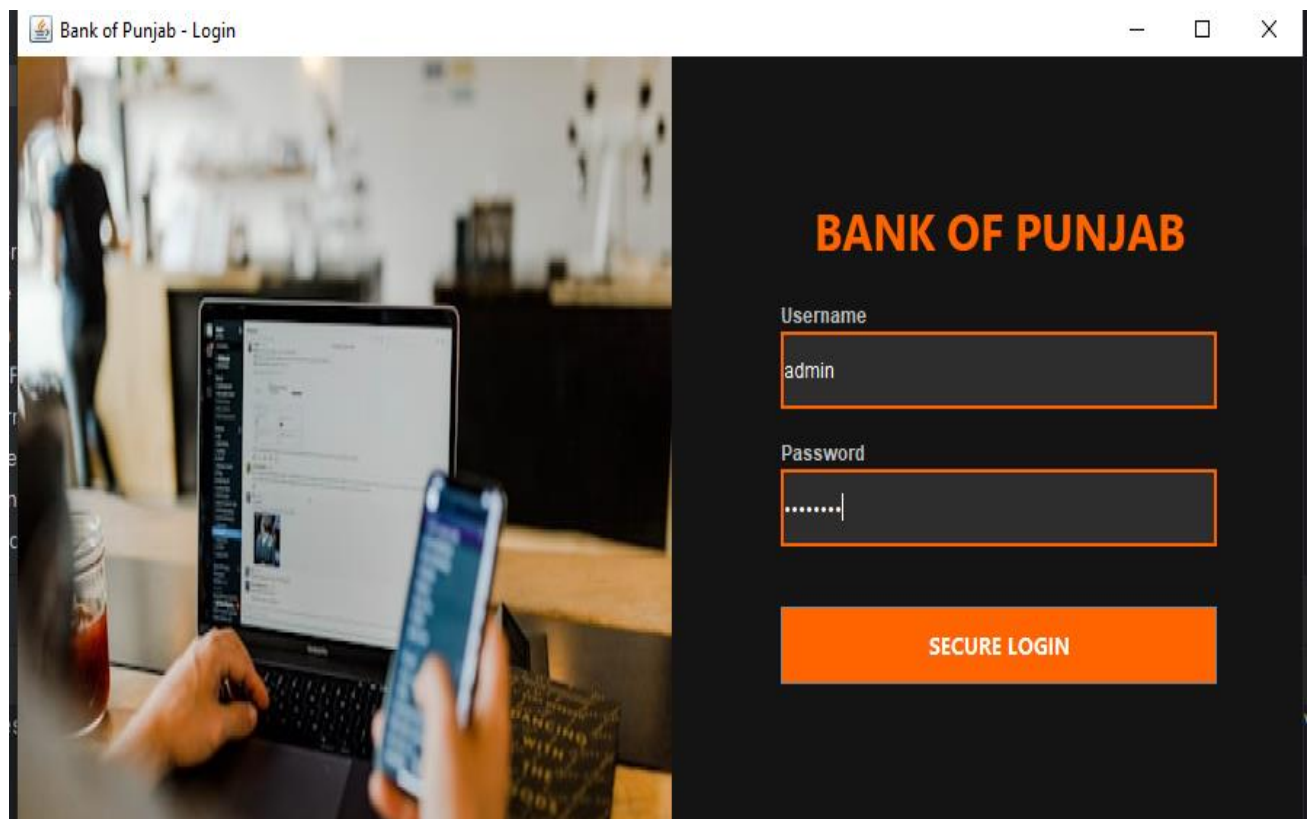
select* from users;

```

Following is the ERD:



Final Look of BANK MANAGEMENT SYSTEM



BOP

Menu ▾

Logout

WELCOME TO BANK OF PUNJAB

Your Trust, Our Commitment.

BOP

Menu ▾

Logout

Client ID:

2

Message



Updated!

OK

12345

SAVE

UPDATE

BOP

Menu ▾

Logout

Account ID:

3

Message

×



Success!

OK

DEPOSIT

BOP

Menu ▾

Logout

Account ID:

3

Message

×



Success!

OK

WITHDRAW

BOP

Menu ▾Logout

RECORD TRANSACTION

Transaction ID (for Update):
2

Account Number:
2

Transaction Type:
Withdraw

Date (YYYY-MM-DD):
2026-09-12

SAVE TRANSACTIONUPDATE

Message
Transaction Updated!
OK

4. Conclusion

The Bank of Punjab Management System successfully integrates a sophisticated Java Swing frontend with a robust MySQL backend. By utilizing advanced database features like Stored Procedures and Triggers, the system ensures high performance and strict data validation. The project meets all academic requirements, including a responsive and attractive UI, secure user logging, and comprehensive transaction management. This application provides a solid foundation for a digital banking platform, ensuring "Your Trust, Our Commitment".